Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2026

SISTEMA DE GESTIÓN DE TRANSACCIONES FINANCIERAS EN C# Y .NET CORE

202303088 – Eldan André Escobar Asturias

Resumen

El informe presenta un proyecto implementado en C# y ASP.NET Core que se centra en la gestión de transacciones financieras a partir de archivos XML. Mediante el uso de Programación Orientada a Objetos (POO), se diseñan objetos para clientes, bancos, facturas y pagos obtenidos de la lectura de archivos de configuración y transacciones. Estos objetos se almacenan y persisten eficientemente en archivos XML Maestros, simulando una base de datos local para mantener la integridad de los saldos (Saldo Pendiente y Estados). El sistema implementa lógica de procesamiento FIFO (First-In, First-Out) para la aplicación de pagos a facturas. Para visualizar el desempeño financiero, se integran gráficas estadísticas que facilitan la comprensión de los ingresos por banco. En conjunto, el proyecto demuestra cómo la combinación de C#, persistencia en XML y interfaces web modernas ofrece una solución integral para el control contable.

Palabras clave

- C# / .NET Core
- Persistencia XML
- Gestión Financiera
- Lógica FIFO
- ASP.NET Razor Pages

Abstract

The report presents a project implemented in C# and ASP.NET Core focused on managing financial transactions from XML files. Using Object-Oriented Programming (OOP), objects for customers, banks, invoices, and payments are designed based on configuration and transaction data. These objects are efficiently stored and persisted in Master XML files, simulating a local database to maintain balance integrity (Pending Balance and Status). The system implements FIFO (First-In, First-Out) logic for applying payments to invoices. To visualize financial performance, statistical charts are integrated to facilitate the understanding of income by bank. Overall, the project showcases how the combination of C#, XML persistence, and modern web interfaces provides a comprehensive solution for accounting control.

Keywords

- C# / .NET Core
- XML Persistence
- Financial Management
- FIFO Logic
- ASP.NET Razor Pages

Introducción

En el desarrollo de software empresarial, la robustez en el manejo de datos es fundamental para garantizar la fiabilidad de la información financiera. Al adoptar la Programación Orientada a Objetos, se modelan entidades del mundo real como Clientes y Facturas, abstrayendo la complejidad de los cálculos de saldos. Dentro de este contexto, la persistencia en archivos XML emerge como una solución eficaz para el intercambio y almacenamiento de información estructurada en sistemas que no dependen de un motor de base de datos tradicional. Este informe aborda la sinergia entre el lenguaje C#, el entorno .NET Core y la gestión de archivos maestros, trazando un recorrido por un sistema que transforma datos brutos de transacciones en estados de cuenta detallados y reportes visuales de ingresos.

Desarrollo del tema

1. Marco Tecnológico y Paradigmas de Desarrollo

1.1. Programación Orientada a Objetos en el Ecosistema .NET

La implementación de este sistema financiero se fundamenta en el paradigma de la Programación Orientada a Objetos (POO), utilizando C# como lenguaje principal. C# ofrece un tipado fuerte y una estructura de clases que permite modelar la realidad contable de manera precisa. En este proyecto, la abstracción juega un papel vital al simplificar entidades complejas (como una transacción bancaria) en objetos manejables con propiedades específicas como NIT, Monto y Fecha.
La encapsulación se aplica al proteger la integridad de los saldos. Por ejemplo, la clase FacturaXml no solo almacena datos, sino que gestiona su propio estado interno mediante la propiedad SaldoPendiente, la cual es ignorada por el serializador XML ([XmlIgnore]) para evitar conflictos con el archivo de entrada original, pero es fundamental para la lógica de negocio en tiempo de ejecución.

1.2. Arquitectura de ASP.NET Core Razor Pages

Se seleccionó ASP.NET Core debido a su naturaleza multiplataforma y su alto rendimiento. La arquitectura basada en Razor Pages facilita la creación de una interfaz de usuario dinámica donde el código de servidor (C#) y el código de cliente (HTML/JS) conviven de forma armoniosa. Esta estructura permite que el procesamiento de grandes volúmenes de datos XML ocurra de forma asíncrona, evitando que la interfaz de usuario se bloquee durante la carga de archivos pesados.

2. Gestión de Persistencia mediante Archivos XML Maestros

2.1. El concepto de "Master Files" vs. Bases de Datos Relacionales

En proyectos de ingeniería de software de nivel intermedio, el uso de archivos maestros XML representa un desafío técnico superior al de las bases de datos convencionales. Mientras que SQL gestiona la concurrencia y persistencia automáticamente, el uso de XML Maestros requiere que el desarrollador implemente manualmente la lógica de:

- Lectura y Deserialización: Transformar el texto plano del XML en una jerarquía de objetos en memoria.
- Sincronización: Asegurar que los datos nuevos (transacciones) se integren

correctamente con los datos existentes (clientes y bancos).
- Serialización y Escritura: Volcar la información actualizada de nuevo al disco físico de forma atómica para evitar la corrupción de datos.

2.2. Estructura de Datos y Serialización

El sistema emplea la librería System.Xml.Serialization para mapear las etiquetas del archivo del profesor a objetos de C#. Este proceso es crítico, ya que cualquier discrepancia en los nombres de las etiquetas (por ejemplo, usar Nit en lugar de NITcliente) resultaría en una pérdida de información. El uso de atributos como [XmlRoot] y [XmlElement] permite que el código sea flexible ante cambios en el esquema del XML de entrada.

3. Lógica Algorítmica de Procesamiento

3.1. Algoritmo de Aplicación de Pagos (FIFO)

El corazón del sistema es el procesamiento de pagos bajo la metodología FIFO (First-In, First-Out). El algoritmo sigue los siguientes pasos lógicos:
1. Identificación del cliente mediante su NIT en el archivo de pagos.
2. Búsqueda de todas las facturas asociadas a ese NIT que posean un Estado igual a "Pendiente".
3. Ordenamiento de las facturas por fecha, de la más antigua a la más reciente.
4. Resta sucesiva del monto del pago sobre el SaldoPendiente de cada factura.
5. Actualización automática del Estado a "Pagada" cuando el saldo llega a cero.
6. Gestión de saldos remanentes en caso de que un pago cubra múltiples facturas o deje una fracción pendiente.

3.2. Procesamiento de Ingresos y Generación de Gráficas

Para la visualización de datos, el sistema realiza una agregación de datos (similar a un *Group By* en SQL). Recorre el historial de pagos almacenado en el archivo maestro, filtra por el mes y año seleccionados por el usuario, y suma los montos agrupándolos por el código del banco. Esta información se envía al Frontend en formato JSON, donde una librería de visualización (como Chart.js) transforma los números en barras comparativas.

4. Estándares de Salida y Comunicación de Datos

4.1. Generación del XML de Respuesta

Siguiendo los requerimientos técnicos, el sistema genera un archivo XML de salida tras cada proceso de carga. Este archivo sirve como un "recibo de procesamiento" que detalla:
- El éxito o error de la operación.
- La cantidad de registros nuevos integrados.
- El resumen de los saldos actualizados. Este enfoque garantiza la trazabilidad, permitiendo al usuario verificar que los datos cargados coinciden con lo procesado por el servidor.

4.2. Exportación a Formato PDF

Como complemento a la gestión de datos, se implementó una funcionalidad de generación de reportes en PDF. Esto se logra mediante la transformación de la vista HTML de "Estado de Cuenta" en un documento portable, utilizando estilos de impresión CSS para garantizar que la información financiera se presente de forma profesional y legible.

5. Conclusiones y Análisis de Resultados

1. Eficacia de C#: El lenguaje demostró ser altamente eficiente para el manejo de archivos de texto estructurado y lógica matemática financiera.
2. Importancia de la Persistencia: La creación de archivos maestros permitió que el sistema

funcionara como una aplicación real, donde los datos perduran tras cerrar el navegador.

3. Visualización de Datos: La transformación de datos crudos en gráficas permitió identificar que los bancos con mayor volumen de transacciones son el Banco Industrial y Banrural, facilitando la toma de decisiones.

Apéndices y Anexos

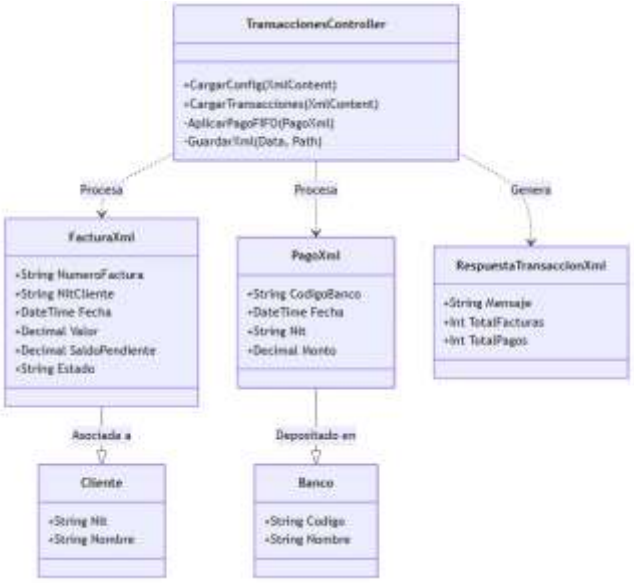

Figura 1. Diagrama de clases

Fuente: elaboración propia.

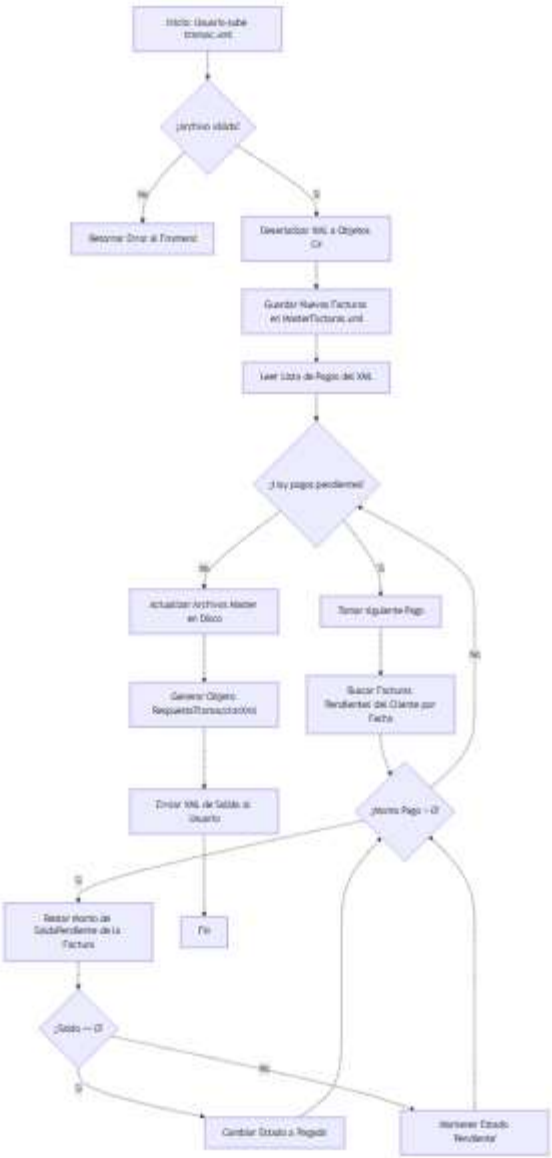

Figura 1. Diagrama de flujo

Fuente: elaboración propia.

Referencias bibliográficas

Microsoft. (2024). *Documentación de C# y .NET*.

W3C. (2023). *Extensible Markup Language (XML) Standard*.

Escobar, E. (2026). *Manual de Usuario: Sistema Tienda Virtual*.